

✓ Week 1 Lab: Introduction to Python

Unit: Fundamentals of AI

Estimated time: 1.5 hours

What you will learn today

By the end of this lab, you should be able to:

- Run Python code in a notebook.
- Use variables, basic data types, lists, tuples, sets, and dictionaries.
- Write simple `if` statements, loops, and functions.
- Use NumPy for numerical arrays.
- Use SciPy for a basic statistical calculation.
- Use pandas to load, inspect, filter, and summarise tabular data.
- Read small pieces of code and explain what they do in plain English.

This notebook is designed for learning by doing. Run each code cell, read the output, then complete the small exercises.

Table of Contents

0. [How to Use This Notebook](#)
1. [Python Syntax: Small Rules That Matter](#)
2. [Variables](#)
3. [Data Types](#)
4. [Comparisons and If Statements](#)
5. [Loops](#)
6. [Functions](#)
7. [Modules and Imports](#)
8. [NumPy: Working with Numbers Efficiently](#)
9. [SciPy: A First Statistical Tool](#)
10. [pandas: Working with Tables](#)
11. [Mini Practice: From Raw Records to a Simple Insight](#)
12. [Short Quiz](#)

✓ 0. How to Use This Notebook

A notebook is made of **cells**.

- Text cells explain ideas.
- Code cells contain Python code.
- To run a code cell in Google Colab or Jupyter, click the cell and press **Shift + Enter**.

Do not worry about memorising syntax today. Your goal is to become comfortable reading, running, and lightly editing Python code.

Good Notebook Habits

- Run cells from top to bottom, because later cells often depend on earlier cells.
- A variable only exists after the cell that creates it has been run.
- If something behaves strangely, try **Runtime > Restart session and run all** in Colab, or **Kernel > Restart & Run All** in Jupyter.
- If you edit an earlier cell, run that cell again before running later cells.
- Save your notebook regularly, especially before trying a larger change.

```
# Run this cell first.  
# Anything after a # symbol is a comment. Python ignores comments.  
  
print("Hello, AI learners!")  
  
Hello, AI learners!
```

Python Comments

Notice how in the code block above certain lines begin with the character **#** and appear in green.

These are **comments** - Python knows to ignore all contents on any line beginning with the special character **#**. We can use this to leave useful notes and comments in our code, that in no way affect how the code actually runs.

Notebooks of course allow us to intersperse cells of text, but comments are still useful!

✓ Exercise 0.1

Change the message in the code cell below so that it prints your own name or study programme.

```
print("My name is Elsa and I am learning Python for AI.")
```

```
My name is Elsa and I am learning Python for AI.
```

✓ 1. Python Syntax: Small Rules That Matter

Python is quite readable, but it is strict about a few things:

- Spelling matters: `print` works, `Print` does not.
- Brackets and quotation marks must be closed.
- Indentation matters, especially inside loops, functions, and `if` statements.

A common programming habit is: **read the error message from the bottom upwards**. The final line often tells you what Python is confused about.

✓ Common Beginner Errors

Errors are normal. They are part of programming, not a sign that you are doing badly.

Error type	Common meaning	Example cau
<code>NameError</code>	Python does not know this name	using a variable before creat
<code>SyntaxError</code>	Python cannot understand the code structure	missing a bracket or quotatic
<code>TypeError</code>	an operation is being used with the wrong kind of data	adding a number to text
<code>IndentationError</code>	spacing at the start of a line is wrong	forgetting to indent after <code>if</code>

When you see an error, read the final line first, then look for the line number mentioned in the message.

```
# This cell shows examples as text, so it will not break when you r

print("NameError example: using patient_count before creating it")
print("SyntaxError example: print('hello' without the closing brack
print("TypeError example: 'age' + 42")
print("IndentationError example: forgetting spaces after an if stat
```

```
# Python can do arithmetic.
```

```
print(2 + 3)
print(10 - 4)
print(6 * 7)
print(20 / 5)
print(2 ** 3) # 2 to the power of 3
```

```
5
6
42
4.0
8
```

✓ Exercise 1.1

A clinic has 18 appointments in the morning and 22 appointments in the afternoon. Use Python to calculate the total number of appointments.

```
# Write your calculation here.
print(18+22)
```

```
40
```

✓ 2. Variables

A **variable** is a name attached to a value.

Think of a variable as a labelled box. The label helps us reuse the value later.

```
patient_age = 54
appointment_type = "follow-up"
cost_gbp = 125.50

print(patient_age)
print(appointment_type)
print(cost_gbp)
```

```
54
follow-up
125.5
```

Variable names should be meaningful. For example, `patient_age` is clearer than `x`.

Python programmers often use **snake_case** for variable names: lower-case words separated by underscores.

```
# We can reuse variables in calculations.

unit_price = 19.99
quantity = 3
total_price = unit_price * quantity

print(total_price)
```

```
59.97
```

✓ Assignment `=` and Equality `==`

Beginners often confuse these two symbols:

- `=` means **put this value into this variable**.
- `==` means **check whether two values are equal**.

Use `=` when creating or updating a variable. Use `==` inside a question or condition.

```
customer_type = "returning" # assignment

print(customer_type == "returning") # equality check
print(customer_type == "new")
```

```
True
False
```

✓ Exercise 2.1

Create three variables:

- `customer_name`
- `items_bought`
- `average_item_price`

Then calculate and print the estimated total spend. You are free to pick your own name & values.

```
# Your code here.
customer_name = "Elsa"
items_bought = "Sweets, fruits"
average_item_price = 3 + 5
print(average_item_price)
```

```
8
```

3. Data Types

A **data type** tells Python what kind of value something is.

Common types:

- `int`: (integer) whole number, such as `42`
- `float`: (floating point) decimal number, such as `42.5`
- `str`: (string) text, such as `"hello"`
- `bool`: (boolean) true/false value, either `True` or `False`
- `list`: ordered, changeable collection, such as `[1, 2, 3]`
- `tuple`: ordered, unchangeable collection, such as `(1, 2, 3)`
- `set`: unordered collection of unique values, such as `{1, 2, 3}`
- `dict`: (dictionary) labelled collection, such as `{"age": 42, "city": "Bristol"}`

```
age = 42
height_m = 1.68
city = "Bristol"
has_completed_consent = True

print(type(age))
print(type(height_m))
print(type(city))
print(type(has_completed_consent))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Strings

A string is text. We can combine strings using `+`, or use an **f-string** to place variables inside text.

```

name = "Amira"
programme = "MSc AI for Business"

message = f"{name} is studying on the {programme} programme."
print(message)

```

Amira is studying on the MSc AI for Business programme.

Lists, Tuples, and Sets

Python has several ways to store multiple values. The differences matter:

Type	Ordered?	Can be changed?	Allows duplicates?	Common use
<code>list</code>	Yes	Yes	Yes	A sequence of values you may edit
<code>tuple</code>	Yes	No	Yes	A fixed record, such as coordinates or a date
<code>set</code>	No	Yes	No	Unique values, such as distinct categories

In simple terms: use a **list** for a flexible sequence, a **tuple** for values that should stay fixed, and a **set** when uniqueness matters.

✓ Lists

A list stores multiple values in order. List positions start at `0`, not `1`. This is important to keep in mind when indexing through a list!

```

weekly_sales = [120, 135, 148, 160, 152]

print(weekly_sales)
print(weekly_sales[0]) # first value
print(weekly_sales[2]) # third value
print(len(weekly_sales)) # "len" is a Python function that can return the length of a list

```

[120, 135, 148, 160, 152]
120
148
5

```
# Lists can be changed.  
  
weekly_sales.append(170)  
print(weekly_sales)  
  
[120, 135, 148, 160, 152, 170]
```

✓ Tuples

A tuple also stores multiple values in order. The key difference is that a tuple is **immutable**, which means it cannot be changed after it is created.

```
appointment_time = (2026, 10, 5, "09:30")  
  
print(appointment_time)  
print(appointment_time[0]) # year  
print(appointment_time[3]) # time  
  
(2026, 10, 5, '09:30')  
2026  
09:30
```

If we tried to change one item inside a tuple, Python would give an error. That can be useful when the values should be treated as fixed.

```
# Uncomment the line below to see the error.  
#appointment_time[3] = "10:00"
```

✓ Sets

A set stores unique values. Sets do not keep duplicate values, and they are not designed for position-based indexing.

```
departments_seen = {"sales", "clinical", "finance", "sales", "finan"  
  
print(departments_seen)  
print(len(departments_seen))  
  
{'sales', 'clinical', 'finance'}  
3
```

```
# Sets are useful when we want the unique values from a list.

survey_locations = ["London", "Cardiff", "London", "Bristol", "Cardiff"]
unique_locations = set(survey_locations)

print(unique_locations)

{'Cardiff', 'London', 'Bristol'}
```

✓ Exercise 3.1

Run the code below, then answer in plain English:

1. Why does `unique_conditions` contain fewer values than `reported_conditions`?
2. Which collection would you use for a row of fixed patient information: a list, tuple, or set?

```
reported_conditions = ["asthma", "diabetes", "asthma", "hypertension", "diabetes"]
unique_conditions = set(reported_conditions)
fixed_patient_summary = ("P001", 67, "hypertension")

print(reported_conditions)
print(unique_conditions)
print(fixed_patient_summary)

['asthma', 'diabetes', 'asthma', 'hypertension', 'diabetes']
{'diabetes', 'asthma', 'hypertension'}
('P001', 67, 'hypertension')
```

✓ Dictionaries

A dictionary stores labelled values as **key-value pairs**. Dictionaries are very common in data work because they can represent one record, one event, or one observation.

```
patient = {
    "patient_id": "P001",
    "age": 67,
    "blood_pressure": 142,
    "has_diabetes": False
}

print(patient)
print(patient["age"])
print(patient["blood_pressure"])
```

```
{'patient_id': 'P001', 'age': 67, 'blood_pressure': 142, 'has_diabet
67
142
```

✓ Exercise 3.2

Create a dictionary called `product` with these keys:

- `product_id`
- `category`
- `price`
- `in_stock`

Then print the product category and price.

```
# Your code here.

product = {
    "product_id":101,
    "category":"shoes",
    "price":299,
    "in_stock": True
}

print(product["category"])
print(product ["price"])
```

```
shoes
299
```

✓ 4. Comparisons and If Statements

AI and data analysis often involve decisions:

- Is this value above a threshold?
- Should this customer be classified as high risk?
- Does this record have missing data?

Python uses comparisons such as:

- `==` equal to
- `!=` not equal to
- `>` greater than
- `<` less than
- `>=` greater than or equal to
- `<=` less than or equal to

```
blood_pressure = 142

if blood_pressure >= 140:
    print("High blood pressure reading")
else:
    print("Reading below high threshold")

High blood pressure reading
```

Notice the indentation after `if` and `else`. Python uses indentation, not brackets, to know which lines belong inside the decision block.

✓ Exercise 4.1

A business uses a simple rule: if a customer spends at least £100, they are labelled as `high_value`. Otherwise, they are labelled as `standard`.

Write an `if` statement for a variable called `customer_spend`.

```
customer_spend = 85

# Your code here.
if customer_spend >= 100:
    print("labelled as high value"),
else:
    print("labelled as standard")
```

```
labelled as standard
```

✓ 5. Loops

A loop repeats code. This is useful when we want to process many records.

The most common beginner-friendly loop is a `for` loop.

Much like with an `if` statement, Python relies on indentation to know which part of code belongs to the loop block.

```
departments = ["marketing", "finance", "operations", "clinical serv

for department in departments:
    print(department)
```

```
# We can use loops to calculate a summary.

patient_wait_times = [12, 18, 7, 25, 15]
total_wait = 0

for wait_time in patient_wait_times:
    total_wait = total_wait + wait_time

average_wait = total_wait / len(patient_wait_times)
print(average_wait)
```

```
15.4
```

✓ Exercise 5.1

The list below contains customer satisfaction scores from 1 to 5.

Use a loop to count how many scores are equal to 5.

```
satisfaction_scores = [5, 4, 3, 5, 2, 5, 4, 5]
number_of_fives = 0
```

```
# Your loop here.
```

```
print(number_of_fives)
```

```
0
```

✓ 6. Functions

A function is a reusable block of code. Functions help us avoid repeating ourselves.

You create a function using the keyword `def`, followed by:

- the function name,
- round brackets `()` for any input values,
- a colon `:`,
- indented code underneath.

The basic pattern is:

```
def function_name(input_value):
    result = input_value * 2
    return result
```

A function can:

- receive input values, often called **parameters** or **arguments**,
- perform a calculation or action,
- return an output value.

To **call** a function, write its name followed by brackets. If the function needs inputs, put the input values inside the brackets.

✓ Creating and Calling a Function

The function below calculates a total price. It has two input values: `unit_price` and `quantity`.

`return total` sends the answer back to the place where the function was called.

```
def calculate_total_price(unit_price, quantity):  
    total = unit_price * quantity  
    return total
```

```
# Calling the function directly inside print.  
print(calculate_total_price(19.99, 3))  
print(calculate_total_price(5.50, 10))
```

```
59.97  
55.0
```

✓ Return Values

A **return value** is the output that comes back from a function.

You can store a return value in a variable and use it later. This is one of the main reasons functions are useful.

```
def calculate_total_price (price, quantity):  
    total= price * quantity  
    return total  
order_total = calculate_total_price(12.50, 4)  
discounted_total = order_total * 0.90  
  
print(order_total)  
print(discounted_total)
```

```
50.0  
45.0
```

✓ `print` and `return` Are Different

- `print()` shows a value on the screen.
- `return` gives a value back from a function so the rest of the program can use it.

For data analysis and AI, `return` is especially important because we often want to pass results from one step to another.

```
def add_vat(price):  
    price_with_vat = price * 1.20  
    return price_with_vat  
  
final_price = add_vat(100)  
print(final_price)
```

120.0

✓ Functions with Decisions

Functions can also contain `if` statements. This is useful when we want to turn a rule into reusable code.

```
def classify_blood_pressure(systolic_bp):  
    if systolic_bp >= 140:  
        return "high"  
    else:  
        return "not high"  
  
print(classify_blood_pressure(142))  
print(classify_blood_pressure(128))
```

high
not high

✓ Exercise 6.1

Write a function called `classify_customer`.

- It should take one input: `spend`.
- If `spend` is at least 100, return `"high_value"`.
- Otherwise, return `"standard"`.

Test it using values `85` and `120`.

```
# Your function here.  
  
def classify_customer(spend):  
    if spend >= 100:  
        return "high_value"  
    else:  
        return "standard"  
print(classify_customer(85))  
print(classify_customer(120))
```

```
standard  
high_value
```

✓ 7. Modules and Imports

Python comes with many useful tools, but not all of them are loaded automatically.

A **module** is a file or package of Python code that gives us extra functions, objects, or tools. A **library** is often a larger collection of modules.

To use a module or library, we usually need to **import** it first. We do this by writing `import` followed by the module name.

`import` statements are typically placed right at the top of our code.

✓ Importing a Built-in Module

Python has built-in modules that are already installed. For example, the `math` module contains common mathematical functions.

```
import math

print(math.sqrt(16))
print(math.pi)
```

```
4.0
3.141592653589793
```

The dot `.` means "look inside this module". So `math.sqrt(16)` means: use the `sqrt` function from inside the `math` module.

This function calculates the square root of a number, passed as the input argument - in our example, we are calculating the square root of 16.

✓ Importing with a Short Name

Some libraries have long names or are used very often. Python lets us give them a shorter alias using `as`.

An **alias** is a short name that points to the imported module. After we create the alias, we use the short name in the rest of the code.

The pattern is:

```
import library_name as short_name
short_name.some_function()
```

You will see these standard aliases throughout AI and data analysis:

- `import numpy as np`: use `np.array()`, `np.mean()`, and other NumPy tools.
- `import pandas as pd`: use `pd.DataFrame()`, `pd.read_csv()`, and other pandas tools.

This does not change the library. It just gives us a shorter name to type. Use the same short names as other Python users where possible, because it makes your code easier for tutors, classmates, and future colleagues to read.

```
import statistics as stat

scores = [72, 81, 69, 90, 78]
average_score = stat.mean(scores)

print(average_score)
```

78

✓ Importing One Tool from a Module

Sometimes we import just one function from a module. This allows us to then call the tool directly, as if it was a declared function, instead of having to type out the module name, followed by dot `.` and then the tool name.

This can make code shorter, but for beginners it is often clearer to import the whole module first.

```
from math import sqrt

print(sqrt(25))
```

5.0

In this unit, we will mostly use imports that are standard in data science, such as NumPy, SciPy, pandas, and later scikit-learn.

✓ 8. NumPy: Working with Numbers Efficiently

NumPy is a Python library for numerical computing. It is widely used in AI because machine learning models often work with large tables or arrays of numbers.

A NumPy array is like a list, but designed for fast mathematical operations.

NumPy is usually imported with the short name `np`.

```
import numpy as np
```

Common NumPy Functions and Attributes

Name	Description
<code>np.array()</code>	Creates a NumPy array from a Python list or other sequence.
<code>np.zeros()</code>	Creates an array filled with zeros. Useful for placeholders.
<code>np.ones()</code>	Creates an array filled with ones.
<code>np.arange()</code>	Creates a sequence of numbers, similar to <code>range()</code> but as a NumPy array.
<code>np.linspace()</code>	Creates evenly spaced numbers between a start and end value.
<code>.shape</code>	Shows the dimensions of an array, such as rows and columns.
<code>.mean()</code> or <code>np.mean()</code>	Calculates the average value.
<code>.std()</code> or <code>np.std()</code>	Calculates the standard deviation.
<code>.min()</code> / <code>.max()</code>	Finds the smallest or largest value.
<code>np.argmax()</code>	Finds the position of the largest value.
<code>np.abs()</code>	Returns absolute values, removing negative signs.

✓ Creating Arrays

The most common way to create a NumPy array is to pass a Python list into `np.array()`.

In the context of this unit - and really, in a broader machine learning context - we tend to use Numpy arrays for numerical data instead of native Python lists almost exclusively.

```
wait_times = np.array([12, 18, 7, 25, 15])

print(wait_times)
print(type(wait_times))

[12 18  7 25 15]
<class 'numpy.ndarray'>
```

We can also create arrays filled with zeros, ones, or a sequence of numbers. These are useful when preparing data for analysis or machine learning.

```
zeros_array = np.zeros(5)
ones_array = np.ones(5)
sequence_array = np.arange(1, 6)

print(zeros_array)
print(ones_array)
print(sequence_array)
```

```
[0. 0. 0. 0. 0.]
[1. 1. 1. 1. 1.]
[1 2 3 4 5]
```

✓ Indexing

Indexing means selecting one value by its position. Like Python lists, NumPy array positions start at **0**.

```
print(wait_times[0]) # first value
print(wait_times[2]) # third value
print(wait_times[-1]) # last value
```

```
12
7
15
```

✓ Slicing

Slicing means selecting a range of values. The value before the colon is where the slice starts. The value after the colon is where it stops, but that stopping position is NOT included in the slice; we take values UP TO that point.

```
print(wait_times[0:3]) # first three values
print(wait_times[2:]) # from position 2 to the end
print(wait_times[:4]) # from the start to before position 4
```

```
[12 18 7]
[ 7 25 15]
[12 18 7 25]
```

✓ Shape

The `.shape` attribute tells us the size of an array. For a table-like 2D array, it tells us the number of rows and columns.

```
patient_measurements = np.array([
    [67, 142, 82],
    [54, 128, 76],
    [72, 150, 88]
])

print(patient_measurements)
print(patient_measurements.shape)
```

```
[[ 67 142  82]
 [ 54 128  76]
 [ 72 150  88]]
(3, 3)
```

For this example, the shape `(3, 3)` means 3 rows and 3 columns. Note the order of dimensions - rows first!

✓ Mean and Standard Deviation

NumPy can quickly calculate summary statistics.

- The **mean** is the average.
- The **standard deviation** describes how spread out the values are.
- **min** and **max** can immediately give us the smallest and largest values.

```
print(wait_times.mean())
print(wait_times.std())
print(wait_times.min())
print(wait_times.max())
```

```
15.4
6.019966777316964
7
25
```

```
# NumPy can apply calculations to all values at once.
```

```
prices = np.array([10.00, 25.00, 40.00])  
prices_with_vat = prices * 1.20
```

```
print(prices_with_vat)
```

```
[12. 30. 48.]
```

This is called **vectorised calculation**. We multiplied every value in the array without writing a loop.

✓ Exercise 8.1

Create a NumPy array called `monthly_revenue` using these values:

```
[2200, 2450, 2100, 2600, 2750, 3000]
```

Then:

1. Print the first value.
2. Print the first three values using slicing.
3. Print the shape.
4. Print the mean, standard deviation, minimum, and maximum revenue.

```
# Your code here.
import numpy as np

monthly_revenue = np.array([2200, 2450, 2100, 2600, 2750, 3000])
print(monthly_revenue [0])
print(monthly_revenue[:3])
print(monthly_revenue.shape)
print("mean:", np.mean(monthly_revenue))
print("standard_deviation:", np.std(monthly_revenue))
print("Minimum:", np.min(monthly_revenue))
print("Maximum:", np.max(monthly_revenue))
```

```
2200
[2200 2450 2100]
(6,)
mean: 2516.6666666666665
standard_deviation: 309.12061651652346
Minimum: 2100
Maximum: 3000
```

✓ 9. SciPy: A First Statistical Tool

SciPy builds on NumPy and provides scientific and statistical tools.

Today we will use one simple example: calculating a z-score.

A **z-score** tells us how far a value is from the mean, measured in standard deviations. This can help identify unusual values, i.e. outliers.

✓ Common SciPy Functions

SciPy is a large library. In this unit, you only need a light introduction. These functions are common in statistics and data analysis:

Name	Description
<code>stats.zscore()</code>	Calculates z-scores, showing how unusual each value is compared to the mean.
<code>stats.describe()</code>	Gives a quick statistical summary, including count, min, max, mean, and standard deviation.
<code>stats.pearsonr()</code>	Calculates correlation between two numerical variables.
<code>stats.ttest_ind()</code>	Compares the means of two independent groups.
<code>stats.normaltest()</code>	Checks whether data looks approximately normally distributed.
<code>scipy.spatial.distance.euclidean()</code>	Calculates Euclidean distance between two points or vectors.

```

from scipy import stats
import numpy as np

lab_results = np.array([4.8, 5.1, 4.9, 5.0, 8.2])
z_scores = stats.zscore(lab_results)

print(z_scores)
print("Most unusual value:", lab_results[np.argmax(np.abs(z_scores))])

[-0.61357199 -0.38348249 -0.53687549 -0.46017899  1.99410897]
Most unusual value: 8.2

```

In later weeks, we will use statistical thinking to understand patterns, uncertainty, and model performance.

✓ Exercise 9.1

Use `stats.zscore()` on the customer spend values below. Which value looks most unusual?

```

customer_spend_values = np.array([45, 52, 48, 50, 500])

# Your code here.
z_scores = stats.zscore(customer_spend_values )
print(z_scores)
print(customer_spend_values )

[-0.52073287 -0.48195489 -0.50411374 -0.49303431  1.99983581]
[ 45  52  48  50 500]

```

✓ 10. pandas: Working with Tables

pandas is one of the most important Python libraries for data analysis. It helps us work with tables, similar to spreadsheets.

In pandas:

- A `DataFrame` is a table.
- A column is one variable, such as age or spend.
- A row is one observation, such as one customer or one patient.

Many pandas tools are used as **methods**. A method is a function that belongs to an object. For example, `customers.head()` calls the `head()` method on the `customers` DataFrame.

You will often see this pattern:

```
table.method_name()
table["column_name"].method_name()
```

Common pandas Functions and Methods

Name	Description
<code>pd.DataFrame()</code>	Creates a DataFrame, usually from a dictionary, list, or other structured data.
<code>pd.read_csv()</code>	Loads a CSV file into a DataFrame.
<code>.to_csv()</code>	Saves a DataFrame as a CSV file.
<code>.head()</code>	Shows the first few rows of a DataFrame.
<code>.info()</code>	Shows column names, data types, and missing-value counts.
<code>.describe()</code>	Gives summary statistics for numerical columns.
<code>.shape</code>	Shows the number of rows and columns.
<code>.mean()</code> / <code>.min()</code> / <code>.max()</code>	Calculates summary values for numerical columns.
<code>.value_counts()</code>	Counts how often each value appears in a column.
<code>.sort_values()</code>	Sorts rows by one or more columns.
<code>.groupby()</code>	Groups rows by a column, then allows summaries for each group.
<code>.isna()</code>	Checks for missing values.

✓ Creating a DataFrame

We can create a DataFrame from a Python dictionary. Each key becomes a column name, and each list becomes the values in that column.

```
import pandas as pd

data = {
    "customer_id": ["C001", "C002", "C003", "C004", "C005"],
    "age": [24, 51, 37, 45, 62],
    "monthly_spend": [45.50, 120.00, 80.25, 220.10, 150.75],
    "clicked_ad": [False, True, False, True, True]
}

customers = pd.DataFrame(data)
customers
```

	customer_id	age	monthly_spend	clicked_ad	
0	C001	24	45.50	False	
1	C002	51	120.00	True	
2	C003	37	80.25	False	
3	C004	45	220.10	True	
4	C005	62	150.75	True	

Next steps:

[Generate code with customers](#)[New interactive sheet](#)

✓ Inspecting a Table

These methods help us quickly understand a table before doing analysis:

- `head()` shows the first few rows.
- `info()` shows column names, data types, and missing-value counts.
- `describe()` gives summary statistics for numerical columns.
- `shape` shows the number of rows and columns. `shape` is an attribute, so it does not use brackets.

You can use `display` to see the entire table printed out in the notebook, but this is rarely practical with large datasets.

```
customers.head()
```

	customer_id	age	monthly_spend	clicked_ad	
0	C001	24	45.50	False	
1	C002	51	120.00	True	
2	C003	37	80.25	False	
3	C004	45	220.10	True	
4	C005	62	150.75	True	

Next steps:

[Generate code with customers](#)[New interactive sheet](#)

```
customers.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customer_id     5 non-null     object
1   age             5 non-null     int64
2   monthly_spend   5 non-null     float64
3   clicked_ad      5 non-null     bool
dtypes: bool(1), float64(1), int64(1), object(1)
memory usage: 257.0+ bytes
```

```
customers.describe()
```

	age	monthly_spend
count	5.000000	5.000000
mean	43.800000	123.320000
std	14.342245	67.157972
min	24.000000	45.500000
25%	37.000000	80.250000
50%	45.000000	120.000000
75%	51.000000	150.750000
max	62.000000	220.100000

```
customers.shape
```

```
(5, 4)
```

✓ Selecting Columns

Use square brackets and the column name to select one column.

```
customers["monthly_spend"]
```

	monthly_spend
0	45.50
1	120.00
2	80.25
3	220.10
4	150.75

```
dtype: float64
```

A selected column also has useful methods, such as `mean()`, `min()`, `max()`, and `value_counts()`.

```
print(customers["monthly_spend"].mean())  
print(customers["monthly_spend"].max())  
print(customers["clicked_ad"].value_counts())
```

```
123.32000000000001  
220.1  
clicked_ad  
True      3  
False     2  
Name: count, dtype: int64
```

✓ Filtering Rows

Filtering means keeping only the rows that match a condition.

```
high_spend_customers = customers[customers["monthly_spend"] >= 100]  
high_spend_customers
```

	customer_id	age	monthly_spend	clicked_ad	
1	C002	51	120.00	True	
3	C004	45	220.10	True	
4	C005	62	150.75	True	

Next steps:

[Generate code with high_spend_customers](#)[New interactive sheet](#)

✓ Sorting Rows

`sort_values()` sorts a table by one column.

```
customers.sort_values("monthly_spend", ascending=False)
```

	customer_id	age	monthly_spend	clicked_ad	
3	C004	45	220.10	True	
4	C005	62	150.75	True	
1	C002	51	120.00	True	
2	C003	37	80.25	False	
0	C001	24	45.50	False	

✓ Grouping and Summarising

`groupby()` splits the table into groups, then we can calculate a summary for each group.

Here, we compare average monthly spend for customers who clicked an advert and customers who did not.

```
customers.groupby("clicked_ad")["monthly_spend"].mean()
```

monthly_spend	
clicked_ad	
False	62.875000
True	163.616667
dtype: float64	

✓ Saving a Table to CSV

A CSV file is a simple table file that can be opened by spreadsheet software and many data tools.

Use `to_csv()` to save a DataFrame. The option `index=False` avoids saving the row numbers as an extra column.

```
csv_path = "customers_week1.csv"
customers.to_csv(csv_path, index=False)

print("Saved table to", csv_path)

Saved table to customers_week1.csv
```

✓ Loading a Table from CSV

Use `pd.read_csv()` to load a CSV file into a new DataFrame.

```
loaded_customers = pd.read_csv(csv_path)
loaded_customers.head()
```

	customer_id	age	monthly_spend	clicked_ad	
0	C001	24	45.50	False	
1	C002	51	120.00	True	
2	C003	37	80.25	False	
3	C004	45	220.10	True	
4	C005	62	150.75	True	

Next steps: [Generate code with loaded_customers](#) [New interactive sheet](#)

✓ Exercise 10.1

Use the `customers` table to answer these questions with code:

1. What is the average age?
2. Which customers clicked an advert?
3. Sort the customers by monthly spend from highest to lowest.
4. Save the table to a CSV file called `my_customers.csv`, then load it back using `pd.read_csv()`.

```
# Your code here.
```

```
average_age = customers["age"].mean()
print ("Average:", average_age)
```

```
clicked_customers = customers[customers["clicked_ad"]]
print(clicked_customers)
```

```
sorted_customers = customers.sort_values(by="monthly_spend", ascending=True)
print(sorted_customers)
```

```
customers.to_csv("my_customers.csv", index=False)
loaded_customers= pd.read_csv("my_customers.csv")
print(loaded_customers)
```

```
Average: 43.8
```

```
customer_id  age  monthly_spend  clicked_ad
1          C002   51          120.00         True
3          C004   45          220.10         True
4          C005   62          150.75         True
customer_id  age  monthly_spend  clicked_ad
3          C004   45          220.10         True
4          C005   62          150.75         True
1          C002   51          120.00         True
2          C003   37           80.25        False
0          C001   24           45.50        False
customer_id  age  monthly_spend  clicked_ad
0          C001   24           45.50        False
1          C002   51          120.00         True
2          C003   37           80.25        False
3          C004   45          220.10         True
4          C005   62          150.75         True
```

✓ 11. Mini Practice: From Raw Records to a Simple Insight

This short task brings together dictionaries, pandas, functions, and basic interpretation.

Imagine a clinic wants to understand whether missed appointments are more common for longer appointments. This is a small toy dataset, not enough for a real clinical conclusion.

```

appointments = [
    {"appointment_id": "A001", "duration_minutes": 20, "attended":
    {"appointment_id": "A002", "duration_minutes": 45, "attended":
    {"appointment_id": "A003", "duration_minutes": 30, "attended":
    {"appointment_id": "A004", "duration_minutes": 60, "attended":
    {"appointment_id": "A005", "duration_minutes": 25, "attended":
    {"appointment_id": "A006", "duration_minutes": 50, "attended":
]

appointments_df = pd.DataFrame(appointments)
appointments_df

```

	appointment_id	duration_minutes	attended	
0	A001	20	True	
1	A002	45	False	
2	A003	30	True	
3	A004	60	False	
4	A005	25	True	
5	A006	50	True	

Next steps:

[Generate code with appointments_df](#)[New interactive sheet](#)

```
appointments_df.groupby("attended")["duration_minutes"].mean()
```

duration_minutes	
attended	
False	52.50
True	31.25
dtype: float64	

✓ Reflection

In plain English, what does the result suggest?

Also consider: why should we be careful about making a strong conclusion from this tiny dataset?

Write your answer here:

✓ 12. Short Quiz

Answer these without running code first. Then use Python to check your answers where helpful.

For extra practice after this lab, you can also try the [W3Schools Python Quiz](#).

Questions

1. What is the output of this code?

```
x = 10
y = 3
print(x + y)
```

2. Which data type would you use for the value "London"?
3. In Python, what position number gives the first item in a list?
4. What does this condition check?

```
monthly_spend >= 100
```

5. What is one reason we use functions?
6. What kind of structure is this?

```
{"age": 45, "city": "Cardiff"}
```

7. Which library is most directly associated with tables/DataFrames?
8. What does `customers.head()` show?
9. Why should we be careful when drawing conclusions from a very small dataset?
10. In your own words, what is one way Python could be useful in business or healthcare?

✓ Optional Code Cell for Checking Quiz Answers

```
# Use this space to test any quiz question with code.
```

Quiz Answers

Try the quiz before reading this section.

1. 13
2. str, because it is text.
3. Position 0.
4. It checks whether monthly_spend is greater than or equal to 100.
5. Functions let us reuse code, make code clearer, and reduce repetition.
6. A dictionary, using key-value pairs.
7. pandas.
8. The first few rows of the DataFrame.
9. A small dataset may not represent the wider population, and patterns may appear by chance.
10. Example answers: analysing sales, predicting demand, identifying patient risk groups, summarising survey data, checking data quality.

End of Lab Checklist

Before you leave, check that you can:

- Run a notebook code cell.
- Explain what a variable is.
- Identify strings, numbers, lists, and dictionaries.
- Use a simple `if` statement.
- Use a `for` loop over a list.
- Write a simple function.
- Create a NumPy array and calculate a mean.
- Use pandas to inspect and filter a table.

You do not need to be fast yet. Comfort comes from repeated practice.